Nnomm Flash-Deal Claim Service

System Architecture, File Navigation, Code Modification Playbook & Staff-Level Technical Interview Guide

Build Status: 16/16 Automated Concurrency Tests Passing (100%) | Stack: Kotlin, Ktor, PostgreSQL, HikariCP, Flyway

> **Repository & Git Status:**
> No remote git push was performed. All commits and code changes exist exclusively in your **local Git repository** on this machine at `e:\Projects\nnomm`. You have 100% control over where and when to push this code.

Table of Contents

1. **0. What is this Project About? (The Real-World Business Problem)**
2. **1. Executive Architecture Overview** (Design Philosophy & High-Level Flow)
3. **2. Complete Codebase Map ("Where is Where")** (Every file explained)
4. **3. The Modification Playbook ("How to Change Things Easily")** (Step-by-step recipes)
5. **4. Concurrency & Data Invariants Deep Dive** (PostgreSQL row locks, MVCC, idempotency state machine)
6. **5. Staff-Level Technical Interview Questions & Answers** (15 detailed questions)
7. **6. Local Testing & Git Instructions**

---

0. What is this Project About? (The Real-World Business Problem)

> **In Simple Terms:**
> Nnomm is a flash-deal food platform (similar to Swiggy Dineout, Zomato Gold, or Groupon). Restaurants publish limited-quantity, heavily-discounted offers (e.g. *"50% off Luxury 5-Course Dinner — Only 10 vouchers available at 8:00 PM!"*).
>
> This repository is the **core, mission-critical reservation & claim backend engine** designed to handle the intense traffic spikes when thousands of customers compete for those 10 units at the exact same millisecond.

The 5 Real-World Catastrophes in Flash Sales (Why this service was built)

Real-World Challenge	The Nightmare Failure (Without this engine)	How this Project Solves It
1. Massive Concurrency & Overselling	At 8:00:00 PM, 5,000 hungry users hit "Claim" for the last 1 remaining burger. Two separate servers read `stock = 1` at the same instant and both sell it! 15 customers arrive at the restaurant with valid vouchers for only 10 seats, causing legal issues, chargebacks, and brand destruction.	**Tuple-Level Row Locking (`FOR UPDATE`) & DB Check Constraints:** PostgreSQL serializes competing requests across all server instances. `CHECK (stock_available >= 0)` physically forbids overselling at the database engine level.
2. Network Flakiness & Double-Charging	A user on a weak 4G connection clicks "Claim". The request succeeds on the server, but the network drops before the confirmation reaches the phone. Panicking, the customer taps "Claim" 5 more times and gets charged 5 times.	**Durable 2-Phase Idempotency:** Every request requires an `Idempotency-Key` . The server recognizes the retry, skips inventory deduction, and replays the original response (`Idempotent-Replayed: true`).
3. Inventory Hoarding & Abandoned Claims	A customer claims the last voucher and is redirected to the payment gateway. They get distracted, close the app, or refuse to pay. If that unit stays locked, the restaurant loses a paying customer.	**Automated Non-Blocking Expiry Worker:** Claims are reserved for 10 minutes. A background worker using PostgreSQL `SKIP LOCKED` sweeps expired claims every 10s and automatically returns inventory to the pool.
4. Millisecond Payment vs. Expiry Race	A customer finishes entering their OTP at 9 minutes and 59 seconds. The payment gateway webhook arrives at the exact same millisecond the background expiry worker sweeps the claim.	**Coordinated State Machine & Row Serialization:** Both processes lock the claim row. If payment wins, claim becomes `PAID` . If expiry wins, claim becomes `REFUND_PENDING` , protecting inventory and customer money.
5. Webhook Spoofing & Replay Attacks	An attacker intercepts a successful payment webhook payload and replays it 100 times to get free meals or forge fake payments.	**HMAC-SHA256 Signatures & Deduplication:** Webhooks require timestamped cryptographic signatures verified in constant time (`MessageDigest.isEqual`) with 5-minute replay expiry and `payment_events` deduplication.

1. Executive Architecture Overview

The Nnomm Flash-Deal Claim Service is engineered to solve the classic distributed systems challenge: **high-concurrency flash sales with zero overselling, zero duplicate claims, sub-millisecond data consistency, and crash resilience across multiple horizontally-scaled instances.**

Core Architectural Principles

- **Database as the Source of Truth:** When multiple stateless service instances process claims concurrently, JVM-level synchronization (`synchronized` , `ReentrantLock` , Kotlin `Mutex`) is completely useless. Serialization is enforced at the database layer via PostgreSQL's tuple-level locks (`SELECT ... FOR UPDATE`) and storage-level check constraints.
- **Zero ORM Overhead:** Rather than using heavy ORMs (like Hibernate/JPA or Exposed) that hide SQL queries, Nnomm uses raw parameterised JDBC prepared statements with HikariCP. Every SQL query is explicit, inspectable, and optimized.
- **Strict 2-Phase Idempotency:** Every mutation requires an `Idempotency-Key` . The service registers an in-progress lock before touching stock, commits the result transactionally, and auto-recovers from node crashes via a 60-second lease timeout.
- **Non-Blocking Expiry Sweeping:** Expired claims are reclaimed by a background coroutine worker utilizing PostgreSQL's `FOR UPDATE SKIP LOCKED` , allowing multiple worker nodes to run simultaneously without lock contention or deadlocks.
- **Constant-Time Security:** Webhooks and JWT tokens are verified using timing-safe operations (`MessageDigest.isEqual`) to prevent side-channel timing attacks.


CLIENT / MOBILE APP
[POST /v1/deals/{id}/claims]
(Idempotency-Key: "k-123")
[POST /v1/webhooks/payments]
(X-Signature: HMAC-SHA256)
ClaimRoutes / AuthHelper
- 16KB payload protection
- Customer JWT auth check
WebhookRoutes / HmacService
- Verify timestamp (<5 min)
- Timing-safe HMAC check
ClaimService
1. Register idempotency lock
2. Check existing / replayed
3. Open DB Transaction:
- SELECT deal FOR UPDATE
- Verify stock & timespan
- Check customer limit
- UPDATE stock_available
- INSERT claim
4. Save response in key table
PaymentWebhookService
1. Deduplicate event_id
2. Lock claim FOR UPDATE
3. Validate amount & status
4. Transition status:
- RESERVED -> PAID
- EXPIRED -> REFUND
- Mismatch -> REVIEW
5. Commit transaction
POSTGRESQL 15
- Tables: deals, claims, idempotency_keys, payment_events
- Engine Level Check Constraints: CHECK(stock_available >= 0)
CHECK(stock_available <= stock_total)
- Unique Indexes: UNIQUE(deal_id, customer_id) WHERE status != 'EXPIRED'
(Sweeps expired claims every 10s)
ClaimExpiryWorker (Background Job)
SELECT * FROM claims WHERE status='RESERVED' AND expires_at <= NOW()
FOR UPDATE SKIP LOCKED
-> Sets status='EXPIRED', re-increments stock_available

2. Complete Codebase Map ("Where is Where")

The project follows a clean layered architecture with zero circular dependencies. Below is the comprehensive guide to every source file in the project.

File Path	Component Layer	Purpose & Exact Responsibilities
`src/main/kotlin/com/nnomm/Application.kt`	Application Entrypoint	Configures and starts the embedded Netty Ktor server on port 8080. Bootstraps the HikariCP database pool, executes Flyway migrations, configures serialization and routing, and launches the background `ClaimExpiryWorker` coroutine.
`com/nnomm/config/AppConfig.kt`	Configuration	Reads environment variables with sensible defaults: `DATABASE_URL`, `DATABASE_USER`, `DATABASE_PASSWORD`, `JWT_SECRET`, `WEBHOOK_SECRET`, `EXPIRY_WORKER_INTERVAL_MS` (default: 10s), `CLAIM_TTL_MINUTES` (default: 10m).
`com/nnomm/database/DatabaseFactory.kt`	Database & Transactions	Manages the HikariCP connection pool and Flyway schema migrations. Provides the transaction helpers `withTransaction { conn -> ... }` and `withConnection { conn -> ... }` with proper auto-commit management and automatic rollback on failure.
`com/nnomm/domain/Models.kt`	Domain & DTOs	Contains all domain entities, enums, and JSON serializable DTOs: `Deal`, `Claim`, `ClaimStatus` (`RESERVED`, `PAID`, `EXPIRED`, `NEEDS_REVIEW`, `REFUND_PENDING`), `UserRole`, `CreateDealRequest`, `ClaimRequest`, `WebhookEvent`.
`com/nnomm/domain/Exceptions.kt`	Domain Exceptions	Standardized exceptions that cleanly map to HTTP error responses: `SoldOutException` (409), `DealNotLiveException` (400), `AlreadyClaimedException` (409), `RequestInProgressException` (409), `DealNotFoundException` (404), `ClaimNotFoundException` (404).
`com/nnomm/repository/DealRepository.kt`	Data Access	Performs raw parameterised SQL queries on `deals`: `findByIdForUpdate` (row locking), `createDeal`, `decrementStock` (atomic check & update), `incrementStock` (reclaiming inventory upon expiry).
`com/nnomm/repository/ClaimRepository.kt`	Data Access	Manages the `claims` table: `createClaim`, `findById`, `findByIdForUpdate`, `findActiveClaimByCustomer`,

File Path	Component Layer	Purpose & Exact Responsibilities
		`updateStatus`, and `findExpiredClaimsForUpdateSkipLocked` (concurrency-safe batch sweeper).
`com/nnomm/repository/IdempotencyRepository.kt`	Data Access	Maintains the `idempotency_keys` table: creates pending lock records with `locked_until` leases, reads existing records for replaying, saves completed HTTP status and response payloads, and renews expired crash locks.
`com/nnomm/repository/PaymentEventRepository.kt`	Data Access	Manages the `payment_events` table: records incoming webhook `event_id`, tracks payment amounts, records outcome reasons, and ensures duplicate webhook events are rejected idempotently.
`com/nnomm/service/DealService.kt`	Business Logic	Validates deal creation parameters (positive price, total stock > 0, start time before end time), verifies admin role privileges, and formats deal responses.
`com/nnomm/service/ClaimService.kt`	Core Concurrency Engine	Orchestrates the entire claim workflow: manages idempotency state machine, acquires row locks on deals, validates deal active time windows, enforces the 1-active-claim-per-customer rule, executes atomic stock decrements, and persists the HTTP 201 response.
`com/nnomm/service/PaymentWebhookService.kt`	Webhook Orchestration	Validates webhook timestamps (5-minute window), verifies HMAC-SHA256 signatures, deduplicates `event_id`, locks the claim row, and executes status transitions (e.g. `RESERVED -> PAID`, `EXPIRED -> REFUND_PENDING`, amount mismatch -> `NEEDS_REVIEW`).
`com/nnomm/service/ClaimExpiryWorker.kt`	Background Worker	A coroutine worker executing periodically. Uses `SKIP LOCKED` to batch-fetch up to 100 expired reserved claims, marks them `EXPIRED`, and atomically restores stock back to the deals.

File Path	Component Layer	Purpose & Exact Responsibilities
`com/nnomm/security/JwtService.kt`	Authentication	Generates and verifies HS256 JWT tokens. Enforces role validations, token expiration, subject extraction, and restaurant association checks.
`com/nnomm/security/HmacService.kt`	Webhook Security	Computes HMAC-SHA256 signatures over `timestamp.raw_body` and validates signatures using constant-time byte comparison (`MessageDigest.isEqual`).
`com/nnomm/routes/DealRoutes.kt, ClaimRoutes.kt, WebhookRoutes.kt`	HTTP Routing	Exposes REST endpoints, validates payload size (16KB ceiling to prevent memory denial-of-service), enforces role authorization, extracts idempotency headers, and routes to appropriate service methods.
`src/main/resources/db/migration/V1__init_schema.sql`	Database Schema	Defines PostgreSQL tables, foreign keys, table-level check constraints (`stock_available >= 0`), partial unique indexes for single-active-claim enforcement, and B-tree indexes for fast queries.

3. The Modification Playbook ("How to Change Things Easily")

If an interviewer or senior engineer asks: "*How would you add feature X or change requirement Y?*", here are the exact steps and files to touch.

Recipe 1: Adding a New Field to a Deal (e.g. `description` or `discount_percent`)

1. **Database Migration:** Create a new Flyway migration file: `src/main/resources/db/migration/V2__add_deal_description.sql`:

```
ALTER TABLE deals ADD COLUMN description TEXT DEFAULT '' NOT NULL;
ALTER TABLE deals ADD COLUMN discount_percent INT DEFAULT 0 CHECK (discount_percent BETWE
```

2. **Update Domain Models:** In `com/nnomm/domain/Models.kt`:
   - Add `val description: String = ""` and `val discountPercent: Int = 0` to the `Deal` data class.
   - Add the fields to `CreateDealRequest` and `DealResponse`.
3. **Update Repository:** In `com/nnomm/repository/DealRepository.kt`:
   - Update `mapRowToDeal(rs: ResultSet)` to read `rs.getString("description")` and `rs.getInt("discount_percent")`.
   - Update the `INSERT INTO deals (...)` SQL in `createDeal()` to include the new columns and bind their values.
4. **Update Service Validation:** In `com/nnomm/service/DealService.kt`, add any business rule checks (e.g. `require(request.discountPercent in 0..100)`).
5. **Done!** Run `./gradlew test` to verify everything compiles and passes.

Recipe 2: Changing the Claim Reservation Expiry Duration

- **Where:** `src/main/kotlin/com/nnomm/service/ClaimService.kt` (line 148).
- **Current logic:** `val expiresAt = now.plusSeconds(config.claimTtlMinutes * 60L)` (reads `AppConfig.claimTtlMinutes` which defaults to 10 minutes).
- **To make it dynamic per deal:**
  1. Add `reservation_ttl_seconds INT DEFAULT 600 NOT NULL` to the `deals` table.
  2. In `ClaimService.kt`, change `expiresAt` calculation to: `now.plusSeconds(deal.reservationTtlSeconds.toLong())`.

Recipe 3: Allowing Multiple Active Claims per Customer for Different Deals

- **Current Rule:** A customer can only hold 1 active (`RESERVED` or `PAID`) claim per deal.
- **Where it is enforced:**

1. **Database Level:** In `V1__init_schema.sql`:

```
CREATE UNIQUE INDEX idx_claims_unique_active_claim
ON claims(deal_id, customer_id)
WHERE status IN ('RESERVED', 'PAID', 'NEEDS_REVIEW');
```

2. **Application Level:** In `com/nnomm/repository/ClaimRepository.kt` inside `findActiveClaimByCustomer()`.

- **To change to "1 claim per customer across the ENTIRE platform":** Change the index definition in SQL to `ON claims(customer_id) WHERE status IN ('RESERVED', 'PAID')`.
- **To allow multiple claims up to a limit (e.g. max 3 units):** Remove the unique index and check `SELECT SUM(quantity) FROM claims WHERE customer_id = ? AND deal_id = ? AND status != 'EXPIRED'` inside the row-locked transaction.

Recipe 4: Adding a New Webhook Event (e.g. `payment.refunded` or `payment.chargeback`)

1. **Domain Status:** In `com/nnomm/domain/Models.kt`, add `REFUNDED` or `CHARGEBACK` to `ClaimStatus` enum if needed.
2. **Webhook Handler:** In `com/nnomm/service/PaymentWebhookService.kt`:

```
when (event.eventType) {
    "payment.succeeded" -> handlePaymentSucceeded(...)
    "payment.failed" -> handlePaymentFailed(...)
    "payment.refunded" -> handlePaymentRefunded(...) // New branch
}
```

3. **Restoring Stock on Refund:** If a refunded claim should return inventory back to the deal, call `dealRepository.incrementStock(conn, claim.dealId, claim.quantity)` within the same transaction.

Recipe 5: Scaling to 100,000 Claims/Sec (Redis Pre-Allocation Architecture)

If an interviewer asks: "*Postgres handles ~3,000 claims/sec on a single beefy instance. How do you scale this to 100,000/sec during a massive Black Friday event?*"

1. **Redis Lua Scripting for Atomic Inventory Check & Decrement:**
   - Load deal inventory into Redis: `deal:{id}:stock = 1000`.
   - Execute a single Redis Lua script that checks `stock >= quantity`, decrements atomically, and records customer claim in a Redis Set (for 1-claim limit). This executes in **0.2 milliseconds in single-threaded Redis memory**.
2. **Asynchronous Kafka Event Queue:**
   - Once Redis accepts the claim, push a `ClaimReservedEvent` to an Apache Kafka topic partitioned by `deal_id`.

    - A consumer service pool reads from Kafka and asynchronously inserts the claim into PostgreSQL in batches.
3. **Eventual Consistency with Strict Bounds:** PostgreSQL is protected from traffic spikes, while Redis acts as the high-throughput throttle guarantee.

4. Concurrency & Data Invariants Deep Dive

The "Last Unit" Race Condition Proof

> **Scenario:** Deal has `stock_available = 1`. Two requests (Req A and Req B) arrive on separate instances at the exact same millisecond.

Step	Request A (Instance 1)	Request B (Instance 2)
1. Connect	Opens DB connection, sets `autoCommit = false`	Opens DB connection, sets `autoCommit = false`
2. Lock Row	Executes `SELECT * FROM deals WHERE id = ? FOR UPDATE`. **PostgreSQL grants ExclusiveLock.**	Executes `SELECT * FROM deals WHERE id = ? FOR UPDATE`. **PostgreSQL suspends Req B into wait queue.**
3. Check & Mutate	Reads `stock_available = 1`. Executes `UPDATE deals SET stock_available = 0`. Inserts claim row with status `RESERVED`.	*Waiting for Row Lock...*
4. Commit	Commits transaction. PostgreSQL releases Row Lock. Returns HTTP 201 Created.	**Row Lock acquired!** Under `READ COMMITTED`, PostgreSQL re-evaluates tuple with newly committed data.
5. Re-check	Completed.	Req B reads updated `stock_available = 0`. Evaluates `stock_available < quantity (0 < 1)`. Throws `SoldOutException`. Rolls back transaction, returns **HTTP 409 SOLD_OUT**.

Crash-Safe Idempotency State Machine

Network failures and client retries are handled through a durable 2-phase idempotency protocol stored in the `idempotency_keys` table:

```
Client Request (Key: "k-123")
      |
      v
Query idempotency_keys WHERE key = "k-123" FOR UPDATE
      |
      +---> Record Found with status_code != NULL:
      |     ==> REPLAY stored response immediately with header "Idempotent-Replayed: true"
      |
      +---> Record Found with status_code == NULL AND locked_until > NOW:
      |     ==> Request currently running on another thread or pod.
      |     ==> Return HTTP 409 REQUEST_IN_PROGRESS
      |
      +---> Record Found with status_code == NULL AND locked_until <= NOW:
      |     ==> Previous pod crashed mid-request!
      |     ==> Renew lock lease: locked_until = NOW + 60s
      |     ==> Re-execute claim transaction safely
      |
      +---> No Record Found:
            ==> Insert new record: locked_until = NOW + 60s, status_code = NULL
            ==> Execute claim transaction: decrement stock, insert claim
            ==> Update idempotency_keys SET status_code = 201, response_body = '...'
            ==> Commit and return HTTP 201 Created
```

Webhook vs Expiry Worker Race Condition Matrix

Race Scenario	Who Wins Lock First	Winner Action	Loser Action	Final State & Stock Invariant
Late Payment Arrives After Expiry	Expiry Worker	Marks claim `EXPIRED`, adds +1 back to `stock_available`. Commits.	Webhook acquires lock, sees status is `EXPIRED`. Transitions status to `REFUND_PENDING`. Does NOT alter stock.	Claim: `REFUND_PENDING`. Stock: Unit remains in stock for others. **Zero inventory lost.**
Payment Arrives Exactly at Expiry	Payment Webhook	Sees status `RESERVED`. Transitions to `PAID`. Keeps stock held. Commits.	Expiry worker acquires lock, sees status is `PAID` (not `RESERVED`). Ignores claim.	Claim: `PAID`. Stock: Remains claimed. **Customer successfully gets deal.**

5. Staff-Level Technical Interview Questions & Answers

These are 15 deep, realistic questions commonly asked at top tech companies (Uber, Amazon, Stripe, DoorDash, Swiggy) for Backend and Distributed Systems roles.

Q1: Why did you use PostgreSQL row-level locks (`FOR UPDATE`) instead of a Kotlin Mutex or Java `synchronized` block?

Answer: Mutexes and synchronization primitives only operate within the memory space of a single JVM process. In production, we run multiple service instances behind a load balancer (as demonstrated in our `docker-compose.yml` running Instance 1 on 8080 and Instance 2 on 8081). Two requests routed to different instances would acquire two separate local mutexes simultaneously, resulting in a race condition and overselling. PostgreSQL provides a single, centralized lock manager in shared memory that coordinates across all instances.

Q2: What is the purpose of `SKIP LOCKED` in the background expiry worker?

Answer: Without `SKIP LOCKED`, if we run multiple instances of the expiry worker, Worker 1 would issue `SELECT ... FOR UPDATE` and lock the first 50 rows. Worker 2 running the same query would block and stall waiting for Worker 1 to finish, causing latency and thread starvation. With `SKIP LOCKED`, PostgreSQL skips any row that is currently locked by another transaction, allowing Worker 2 to immediately pick up the next batch of rows without waiting or deadlocking.

Q3: How do you prevent webhook replay attacks?

Answer: We enforce three independent layers of defense:
1. **Timestamp Check:** The webhook header contains a Unix timestamp. We reject any request where `|now - timestamp| > 300 seconds` (5 minutes). This prevents attackers from replaying captured packets later.
2. **HMAC Signature:** The payload signature is computed over `timestamp + "." + rawBody` using HMAC-SHA256. An attacker cannot tamper with the timestamp without invalidating the signature.
3. **Database Deduplication:** We record every processed `event_id` in the `payment_events` table. If the same `event_id` arrives again, our unique constraint catches it and we return HTTP 200 without re-processing.

Q4: Why do you return HTTP 404 instead of HTTP 403 when a customer requests a claim belonging to someone else?

Answer: This is a crucial security practice known as preventing *Resource Enumeration and Information Disclosure*. If we returned 403 Forbidden, an attacker could iterate through UUIDs to discover which claim IDs exist on our platform. Returning 404 Not Found ensures that an unauthorized user cannot distinguish between a claim that does not exist and a claim belonging to another customer.

Q5: What happens if the server crashes after decrementing stock and creating the claim, but before returning the HTTP 201 response to the client?

Answer: The claim and the HTTP response status (201) are saved in the database within the *same atomic transaction*. When the client's network library inevitably times out and retries with the same `Idempotency-Key`, the service queries `idempotency_keys`, finds the completed record, and replays the stored 201 Created response with the header `Idempotent-Replayed: true`. No new claim is created, and no inventory is deducted twice.

Q6: What happens if the server crashes *during* the transaction before committing?

Answer: When the server process or container terminates, the TCP connection to PostgreSQL drops. PostgreSQL detects the socket disconnect and automatically issues a `ROLLBACK` on any open transaction. Stock is not decremented, and no claim is created. Furthermore, the `idempotency_keys` row has a `locked_until` lease of 60 seconds. During those 60 seconds, retries receive 409 Conflict (`REQUEST_IN_PROGRESS`). Once 60 seconds pass, the expired lock is renewed and the retry executes cleanly.

Q7: Why did you choose raw JDBC over an ORM like Hibernate or Exposed?

Answer: For high-concurrency systems, predictability of SQL execution is paramount:
1. ORMs frequently introduce N+1 query problems and automatic dirty-checking flushes that can hold database row locks longer than necessary.
2. Explicit constructs like `FOR UPDATE SKIP LOCKED` and database check constraints are first-class in raw SQL.
3. Performance: Raw JDBC prepared statements with HikariCP connection pooling have virtually zero CPU and memory allocation overhead compared to reflection-heavy ORM entity managers.

Q8: Why is `MessageDigest.isEqual` used in HMAC verification instead of standard string equality (`==`)?

Answer: Standard string equality operators (like `strA == strB` or `.equals()`) exit early on the first mismatched character. An attacker measuring request response times with high-precision network probes can determine how many leading bytes of their forged signature matched, allowing them to brute-force the signature byte-by-byte (a *timing attack*). `MessageDigest.isEqual` executes in constant time regardless of where the mismatch occurs, neutralizing timing attacks.

Q9: What database isolation level does the service use, and why is it sufficient?

Answer: We use PostgreSQL's default `READ COMMITTED` isolation level. While `READ COMMITTED` is vulnerable to non-repeatable reads in ordinary queries, we use explicit pessimistic locking (`SELECT ... FOR UPDATE`) on both the deal row and the claim row. In PostgreSQL, `FOR UPDATE` locks the row tuple directly, forcing concurrent transactions to serialize. This gives us strict serializability on mutating paths without paying the high abort-and-retry cost of `SERIALIZABLE` isolation level.

Q10: Why do we have database-level check constraints (`CHECK (stock_available >= 0)`) if the application code already checks stock?

Answer: Defense-in-depth. Application code can have bugs, developer oversights, or unexpected edge cases during schema refactoring. A database check constraint is an inviolable mathematical invariant enforced by the storage engine itself. If any query ever attempts to set `stock_available` below zero, PostgreSQL immediately aborts the transaction with an error, preventing catastrophic inventory overselling under any circumstance.

Q11: How do you handle lazy claim expiration on `GET /v1/claims/{id}` ?

Answer: The background expiry worker runs on a 10-second interval. It is possible that a claim passes its `expires_at` timestamp 2 seconds before the worker's next tick. If the customer calls `GET /v1/claims/{id}` during this 2-second window, the brief requires that the API must never display it as `RESERVED` . In `ClaimRoutes.kt` , when reading the claim, if `status == RESERVED && Instant.now().isAfter(expiresAt)` , the API dynamically maps the returned status in the JSON response to `EXPIRED` .

Q12: How would you prevent a distributed denial-of-service (DDoS) attack targeting the claim endpoint?

Answer:
1. **Payload Limiter:** We restrict incoming request bodies to 16KB max using our `PayloadLimitPlugin` .
2. **Rate Limiting at Edge/API Gateway:** Implement Token Bucket or Leaky Bucket algorithms (e.g. via Cloudflare or AWS API Gateway / Kong) based on IP address and Customer ID (e.g. max 5 requests per second per IP).
3. **Fast-Path Cache Validation:** In front of the database, check an in-memory Redis boolean flag `deal:{id}:is_active` . If the deal has already ended or is marked sold out, reject incoming requests immediately at the edge without querying PostgreSQL.

Q13: Why is the payment amount verified against `deals.price_paise * quantity` in the webhook?

Answer: Payment gateways can occasionally be misconfigured, or a malicious client might have altered the price on the frontend before calling the payment SDK. If the webhook reports a paid amount less than the required amount, we transition the claim status to `NEEDS_REVIEW` rather than granting the deal. This alerts human operations to audit the discrepancy while preventing inventory loss.

Q14: How does the system handle database connection pool exhaustion under load?

Answer: We use HikariCP configured with bounded parameters: `maximumPoolSize = 20`, `connectionTimeout = 5000ms`. If all connections are occupied, incoming requests wait up to 5 seconds before failing fast with a 503 Service Unavailable rather than hanging indefinitely. More importantly, our transactions are designed to do only pure SQL operations—no network calls, webhook dispatches, or heavy computations are ever made while holding an active database connection.

Q15: How were the tests written to guarantee concurrency correctness?

Answer: We use embedded real PostgreSQL with Kotlin Coroutines `async(Dispatchers.IO)`. In `ConcurrencyTest.kt`, we fire 200 simultaneous coroutine workers claiming a 50-unit deal concurrently using `awaitAll()`. We verify that exactly 50 succeed, 150 receive 409 Sold Out, and the database `stock_available` ends at exactly 0. We also test 25 simultaneous requests with the exact same `Idempotency-Key`, confirming that exactly one claim is created.

---

6. Local Testing & Git Push Instructions

Automated Test Suite Summary

The test suite runs against a fully-functional embedded PostgreSQL instance, validating all concurrency, authorization, idempotency, and webhook behaviors:

```
$ ./gradlew test --no-daemon

LifecycleAndExpiryTest > testDealNotLiveBeforeOrAfter() PASSED
LifecycleAndExpiryTest > testClaimExpiryReclaimsUnitsAndHidesReservedStatus() PASSED
LifecycleAndExpiryTest > testReplayedConflictStaysConflict() PASSED
LifecycleAndExpiryTest > testIdempotencyReusedWithDifferentParameters() PASSED
AuthAndVisibilityTest > testRoleRestrictionsOnEndpoints() PASSED
AuthAndVisibilityTest > testClaimVisibilityRestrictions() PASSED
AuthAndVisibilityTest > testJwtTokenSecurity() PASSED
ConcurrencyTest > test200ConcurrentClaimsNeverOversell() PASSED
ConcurrencyTest > testPaymentAndExpiryRaceLeavesStockInvariantIntact() PASSED
ConcurrencyTest > testConcurrentRequestsWithSameIdempotencyKey() PASSED
ConcurrencyTest > testConcurrentClaimsBySameCustomerWithDifferentKeys() PASSED
WebhookTest > testBadSignatureRejected() PASSED
WebhookTest > testPaymentAmountMismatchMovesToNeedsReview() PASSED
WebhookTest > testPaymentFailedDoesNotChangeStatus() PASSED
WebhookTest > testStaleTimestampRejected() PASSED
WebhookTest > testDuplicateEventsAreIdempotent() PASSED

BUILD SUCCESSFUL: 16 actionable tests, 16 passed (100% pass rate)
```

Git Status & How to Push to GitHub / GitLab

> **No remote repository is currently attached:**
> All work is committed to your local Git history. To push to your GitHub or GitLab account:
>
> ```
> # 1. Create a new empty repository on GitHub / GitLab (e.g. "nnomm-flash-deal")
> # 2. Open PowerShell or Terminal in e:Projects
> nomm
> # 3. Add your remote URL:
> git remote add origin https://github.com/<your-username>/<your-repo-name>.git
>
> # 4. Push your branch:
> git branch -M main
> git push -u origin main
> ```

Running the Live Service with Multi-Instance Docker Compose

```
# Build and start PostgreSQL + Instance 1 (8080) + Instance 2 (8081)
docker compose up --build

# In a separate terminal, test health:
curl http://localhost:8080/health
curl http://localhost:8081/health
```
